



# Transformers

## 1.) Word Embedding:

convert the input tokens into numerical values.

## 2.) Positional Encoding:

Keep track of word is super important!

eg: "Man eats pizza"

"Pizza eats Man"

(The context is entirely different here)

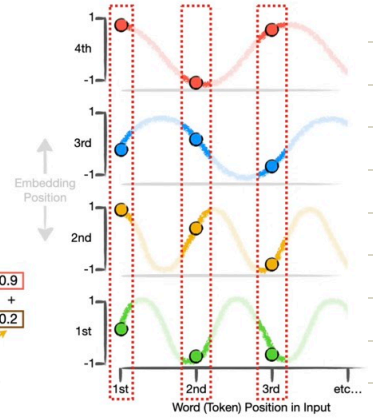
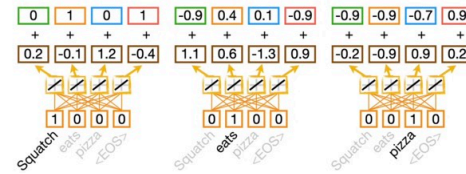
Positional Encoding is the technique where transformer use to keep track of word order.

"Once Embeddings are created for input, The vectors corresponds to "word order" will be added to embeddings to keep track of word order. The values in positional vectors are obtained from sequence of alternating sine and cosine squiggles."

"Positional Encoding keeps track of Word Order"



Thus, each input word ends up with a unique sequence of position values.



## 2.) Self-Attention:

Transformers use Attention Mechanisms to find relationships among the words.

In research paper, it used self attention to capture relationship among the words & itself.

"The pizza came out of the oven and it tasted good"

The diagram shows curved arrows originating from the word 'it' and pointing to 'pizza', 'oven', 'and', 'tasted', and 'good', illustrating the self-attention mechanism where the model captures relationships between the word and other words in the sentence.

Here "pizza" and "it" would have more similarity than "oven" and "it"

In transformer, each token generates three vectors  $Q, k, V$ . The Query Vector represents what information it looks for, The key vector represents what information each token has and Value vector holds the actual content.

The  $Q \times k \Rightarrow$  gives you the attention weights  
(how much percentage of info matches)

Then  $(Q \times k) V \Rightarrow$  gives you the context aware.  
In other words, percentage of info retrieved.

## Analogy

Flexon training.

Prashant studied Concepts (ML, DL, Stats, Time Series).

Prashant has query for the class who finished ML.

Prashant has its own query, Key, Values. Now Prashant's query is ML which will be asked other students.

The other student's key who has ML will have high similarity score.  $\Rightarrow$  Softmax (0 to 1) how much other student's match for prashant query. Now multiply their value to get context of them (or) knowledge is shared.

'Attention All you need' uses 8 heads of attention layer to capture different context / relationships. weights are

randomly initialized initially".

Now we will have 8 outputs of attention Layers but the feed forward Neural Network expects only one matrix.

So concatenate them into single matrix and multiply by  $W^O$  (projection matrix)

8 into 1  
attention head

$$Z = \begin{matrix} Z_0 & Z_1 & \dots & Z_8 \end{matrix} \begin{matrix} \times \end{matrix} \begin{matrix} W^O \\ \text{(Projection Matrix)} \end{matrix} = \begin{matrix} \text{attention head} \end{matrix}$$

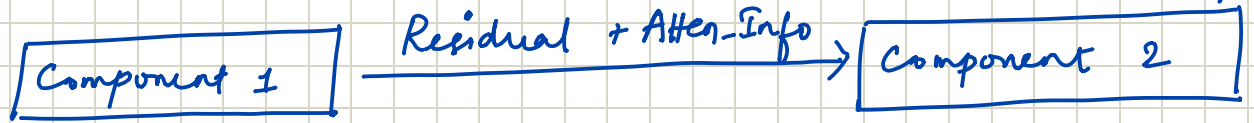
### 3. Residual Connections:

Once the self attention computed  $Z$ , then we add positional encoded values to  $Z$  (self attention values).

These bypasses called Residual Connections which helps to train Neural Network by allowing the self attention layers to establish relationship among the input token without have to preserve word embedding & positional information.

The initial input given might be lost in later stacked layers (vanishing) or may explode (exploding gradient), so adding pos-enc embeddings helps to remember the context from earlier to latest layer by adding it after each layer (component). plus Layer Norm avoids gradient probs.

Eg:



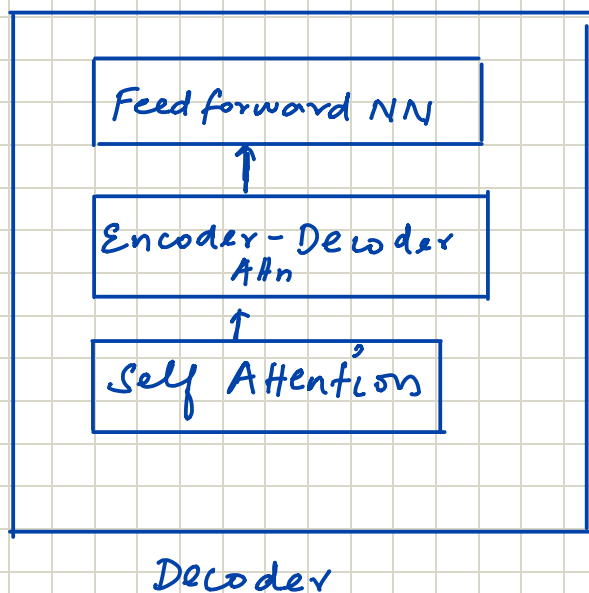
Component 2 gets info about the input plus new context from Component 1.

#### 4.) Layer Normalization:

This helps the stacked layer not fall into any of the gradient problems and also helps in converge faster.

## Decoder

The output of the top encoder is transformed into  $K$  and  $V$  vectors. These vectors to be used in "encoder-decoder attention" which helps decoder focus on appropriate places of input sequence.



Like we did (Add + LN) for every input before Components in encoder, the same process repeated here in decoder.

The self attention Layers work slightly different in Decodes. The self attention layer is allowed only to attend the earlier tokens not the future tokens. (Setting them  $-\infty$ ) before softmax in the self attention.

The Encoder-Decoder Attn ~~is~~ works like multihead attention but uses  $Q$  vector from the decoder and uses  $K$  and  $V$  from output of encoder.

## Final Linear Layer & Normalization:

The decoder stack outputs vectors of floats.

The Linear Layer is simple fully connected Neural Network that projects vectors from decoder stack into much much bigger vectors called logits vector.

The logits vector size would be vocabulary size where each has its own score.

The Softmax layer turns these scores into probabilities (all positive (0 to 1)). The highest probability cell is chosen and output the corresponding word as output at this step.

## Sampling & Temperature

Temperature  $> 1 \Rightarrow$  Creativity

Temperature  $< 1 \Rightarrow$  deterministic

Sampling:

Top  $k \rightarrow$  returns top  $k$  words

Top  $p \rightarrow$  makes high probability items till sums up to  $p$ .



## Types of Attention:

### (i) Self Attention:

Every token attends to other token & itself. It is used in transformer.

Pros: Captures Global Context

Cons:  $O(n^2)$

### (ii) Multiple Query Attention:

Keep multiple Queries but have one K and V shared across multiple heads.

### (iii) Grouped Query Attention:

A middle ground for MHA & MQA.

Group heads (g query heads share 1 K/V vectors)

Eg: 32 heads  $\rightarrow$  4 K/V.

### (iv) Sliding Window Attention:

Designed for long sequence efficiency;